Simulating the Foucault Pendulum: Earth's Rotation in Action

Hongyi Ding

Overview

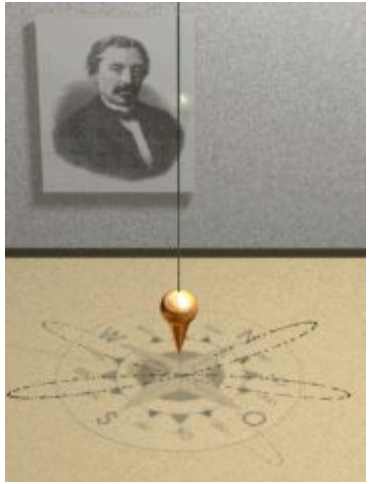
- Introduce the Foucault Pendulum as evidence of Earth's rotation
- Derive the equations of motion in a rotating frame
- Convert the system into first-order form
- Implement numerical integration in Python
- Compare pendulum behavior at different latitudes

Figure: Foucault Pendulum of Griffith Observatory,LA

Brief Background

Ω≈7.29×10−5 rad/s(Earth's angular rotation rate)

The Foucault Pendulum is an experiment that proves the rotation of Earth. It was first introduced in 1851 by the French physicist Léon Foucault.



$$\omega_{precession} = \Omega \sin(\phi)$$

Figure 1. Foucault Pendulum experiment (1851)
Source: Wikipedia

If Earth does not rotate, what will happened?

Equations of Motion in a Rotating Frame

1. Based on Newton's second law
2. $\ddot{x}, \ddot{y} \rightarrow$ acceleration
3. $\dot{x}, \dot{y} \rightarrow$ velocity
4. Added Coriolis coupling(velocity-dependent terms)
5. Simple harmonic motion

$$\ddot{x} - 2\Omega_z \dot{y} + \omega_0^2 x = 0$$
$$\ddot{y} - 2\Omega_z \dot{x} + \omega_0^2 y = 0$$

Equation: Added Coriolis Effect + Newton's second law

Converting to First-Order System

Now, we have to turn the equation into computation that computer can understand. We should turn it into first-order equation instead of second-order, so we can model it.

$$\dot{x} = v_x$$

$$\dot{y} = v_y$$

$$\dot{v}_x = -\omega_0^2 x + 2\Omega \sin\phi v_y$$

$$\dot{v}_y = -\omega_0^2 y - 2\Omega \sin\phi v_x$$

Equation: First-order system

New Library

%matplotlib inline -> make matplotlib inside the notes

matplotlib.animation -> key to create animation

Ipython.display -> embeds the animation as HTML inside the notebook

```
%matplotlib inline
import numpy as np
from matplotlib import pyplot as plt
import matplotlib.animation as animation
from IPython.display import HTML
```

Define the constants

Physics Parameters: gravitational acceleration, pendulum length and natural angular frequency

Earth Rotation: Earth's rotation rate, latitude-dependent rotation component and amplifies rotation for visualization

Numerical Parameters: time resolution, total simulation time and number of iterations

```
# Constants
g = 9.8
L = 10.0
omega0 = np.sqrt(g / L)

Omega_earth = 7.2921159e-5  # rad/s
OMEGA_SCALE = 5000         # amplify rotation for visibility
Omega = Omega_earth * OMEGA_SCALE

dt = 0.01
T = 100
steps = int(T / dt)

latitudes = [0, 42, 60]  # degrees
```

Numerical Integration

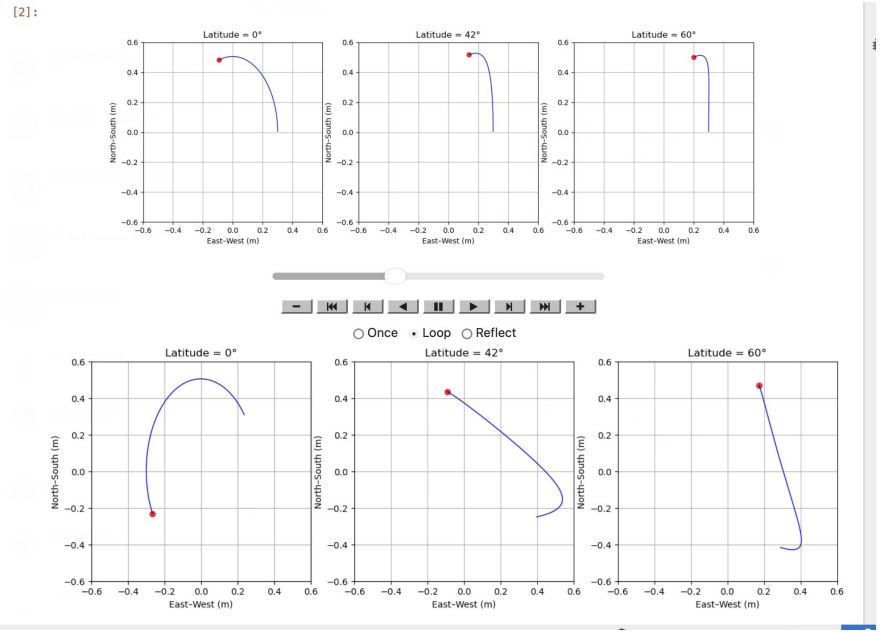
1. Compute acceleration
2. Update velocity
3. Update position
4. Repeat for eachs step

```
for idx, lat in enumerate(latitudes):
    x, y, vx, vy = states[idx]

    phi = np.deg2rad(lat)
    Omega_eff = Omega * np.sin(phi)

    # Foucault equations (Euler integration)
    ax = -omega0**2 * x + 2*Omega_eff * vy
    ay = -omega0**2 * y - 2*Omega_eff * vx

    vx += ax * dt
    vy += ay * dt
    x += vx * dt
    y += vy * dt
```

Animation Implementation

1. Control frame rate and playback
2. Create animation loop
3. Render animation as HTML (since JupterNotes limits the animation size)

```
anim = animation.FuncAnimation(
    fig, animate, init_func=init,
    frames=range(0, steps, 10),
    interval=20, blit=True
)

HTML(anim.to_jshtml())
```

Results and Interpretation

It visually demonstrates how Earth's rotation manifests differently at different latitudes.